

7강. 비동기 처리(fetch/axios)



javascript 비동기 처리

• 비동기 처리

자바스크립트는 싱글 스레드 방식으로 처리해야 할 기능을 순차적으로 처리함.
비동기 처리는 프로그램에서 처리해야 할 기능들을 순차적으로 처리하는 것이 아니라, 원하는 순서에 맞게 처리하는 것

방식	버전	기능
콜백 함수	기존부터 사용	함수 안에 또 다른 함수를 매개변수로 넘겨서 실행 순서를 제어함
프로미스(Promise)	에크마스크립트 2015	프로미스 객체와 콜백 함수를 사용해서 실행 순서를 제어함
async/await	에크마스크립트 2017	async와 await를 사용해서 실행순서 제어

javascript 비동기 처리

- 동기 처리 – async01.js

```
function displayA() {  
  console.log('A');  
}  
  
function displayB() {  
  // console.log('B');  
  // 2초 후에 'B'를 출력하는 비동기 처리  
  setTimeout(() => {  
    console.log('B');  
  }, 2000);  
}  
  
function displayC() {  
  console.log('C');  
}  
  
displayA();  
displayB();  
displayC();
```

/*싱글 스레드이므로 동기적으로 실행됨
displayA() -> displayB() -> displayC() 순서로 실행됨
displayB()는 2초 후에 'B'를 출력하는 비동기 처리이므로,
displayC()가 먼저 실행되어 'C'가 출력됨.
'A' -> 'C' -> 'B' 순서로 출력됨*/

A
C
B

javascript 비동기 처리

- 비동기 처리 – async02.js

```
function displayA() {  
  console.log('A');  
}
```

```
function displayB(callBack) {  
  setTimeout(() => {  
    console.log('B');  
    callBack();  
  }, 2000);  
}
```

```
function displayC() {  
  console.log('C');  
}
```

```
displayA();  
displayB(displayC);
```

```
/*  
비동기 처리 - 프로그램에서 처리해야 할 기능들을 순차적으로  
처리하는 것이 아니라, 동시에 처리하는 방식  
원하는 순서에 맞게 처리하기 위해 콜백함수를 사용  
'A' -> 'B' -> 'C' 순서로 출력됨  
*/
```

A
B
C

javascript 비동기 처리

- 비동기 처리 – async03.js

```
async function displayA() {  
  console.log('A');  
}  
  
async function displayB() {  
  // Promise 객체를 반환하여 비동기 작업 처리  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      console.log('B');  
      resolve();  
    }, 2000);  
  });  
}
```

```
/*  
  비동기 처리 - async/await 방식  
  async/await를 사용하여 더 깔끔하게 비동기 작업 처리  
  Promise를 명시적으로 사용하지 않음  
  'A' -> 'B' -> 'C' 순서로 출력됨  
*/
```

javascript 비동기 처리

- 비동기 처리 – async03.js

```
async function displayC() { // async 함수로 변경
  console.log("C");
}

async function runSequence() { // async 함수로 변경
  displayA();
  await displayB(); // displayB가 완료될 때까지 기다림
  await displayC(); // displayC가 완료될 때까지 기다림
}

runSequence(); // A -> B -> C 순서로 출력됨
```

리액트에서 비동기 처리

■ 리액트에서 비동기 처리

작업이 끝날때 까지 기다리지 않고 다음 코드를 먼저 실행하는 방식

예)

- 서버에서 데이터 가져오기 (API 호출)
- 파일 업로드
- 로그인 요청

■ 기본 동작 흐름

1. 컴포넌트 렌더링
2. useEffect 실행
3. API 요청 (fetch / axios)
4. 데이터 받아오기
5. useState로 저장
6. 화면 다시 렌더링

리액트에서 비동기 처리

▪ useEffect()를 사용하는 이유

```
useEffect() => {  
  // API 호출  
}, [ ]);
```

- 컴포넌트가 처음 렌더링될 때 실행됨
 - 무한 호출 방지
- “화면 뜨자마자 데이터 가져오기” 역할

▪ 상태와 비동기의 관계

```
const [data, setData] = useState([]);
```

서버에서 받은 데이터를 저장하면:

- setData 실행
- 컴포넌트 자동 재렌더링
- 화면 업데이트

리액트에서 비동기 처리

fetch() 와 axios() 차이

항목	fetch	axios
기본 제공 여부	브라우저 내장 API	외부 라이브러리 (설치 필요)
설치	❌ 필요 없음	✅ <code>npm install axios</code>
데이터 변환	<code>response.json()</code> 직접 호출	자동 JSON 변환
응답 데이터 접근	<code>res.json()</code>	<code>res.data</code>
에러 처리	HTTP 에러 자동 처리 ❌	HTTP 에러 자동 throw ✅

리엑트에서 비동기 처리

▪ json Placeholder

JSONPlaceholder는 가짜 데이터가 필요할 때 언제든지 사용할 수 있는 무료 온라인 REST API입니다

{JSON} Placeholder

Free fake and reliable API for testing and prototyping

Powered by [JSON Server](#) + [LowDB](#).

Serving ~3 billion requests each month.

```
fetch('https://jsonplaceholder.typicode.com/todos/1')  
  .then(response => response.json())  
  .then(json => console.log(json))
```

스크립트 실행

```
{  
  "userId": 1,  
  "id": 1,  
  "title": "delectus aut autem",  
  "completed": false  
}
```

리엑트에서 비동기 처리

▪ REST API란?

REST API(Representational State Transfer API)는 웹에서 서버와 클라이언트가 데이터를 주고받기 위한 표준적인 방식입니다.

예를 들어, 웹사이트나 모바일 앱에서 회원 정보, 게시글, 상품 목록 등을 조회하거나 저장할 때 REST API를 사용합니다.

리소스(Resource)를 URL로 표현하고, 그 리소스에 대한 행위는 HTTP 메서드로 구분한다” 개념이다.

리엑트에서 비동기 처리

▪ REST API란?

REST API는 웹에서 사용하는 HTTP를 기반으로 동작합니다.

주요 HTTP 메서드

메서드	기능
GET	데이터 조회
POST	데이터 생성
PUT	데이터 전체 수정
PATCH	데이터 일부 수정
DELETE	데이터 삭제

리엑트에서 비동기 처리

▪ json 데이터

```
[
  {
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },
  {
    "userId": 1,
    "id": 2,
    "title": "quis ut nam facilis et officia qui",
    "completed": false
  },
  {
    "userId": 1,
    "id": 3,
    "title": "fugiat veniam minus",
    "completed": false
  },
  {
    "userId": 1,
    "id": 4,
    "title": "et porro tempora",
    "completed": true
  },
]
```

json(javascript object notation)

- Json이란?

JSON (JavaScript Object Notation) 은

→ 데이터를 문자열 형태로 표현하는 규칙(포맷) 입니다.

- 기본 구조

JSON은 키(key)와 값(value) 으로 이루어져 있습니다.

```
{  
  "name": " 이순신 ",  
  " age " : 45,  
  "isStudent": false  
}
```

"name" → key (항상 문자열)

"홍길동" → value

json(javascript object notation)

■ 배열

```
{  
  "fruits": ["사과", "바나나", "포도"]  
}
```

■ 객체

```
{  
  "product": {  
    "name": "마우스",  
    "price": 30000  
  }  
}
```

■ Json 사용

- 서버 ↔ 클라이언트 데이터 통신
- API 응답 데이터 형식
- 설정 파일 (config)
- 데이터 저장 (localStorage 등)

fetch()를 사용한 비동기 처리

■ 할 일 데이터 전체 가져오기

할 일 (To-do) 테이블

delectus aut autem

quis ut nam facilis et officia qu

fugiat veniam minus

et porro tempora

laboriosam mollitia et enim quasi adipisci quia provident illum

qui ullam ratione quibusdam voluptatem quia omnis

illo expedita consequatur quia in

quo adipisci enim quam ut ab

molestiae perspiciatis ipsa

illo est ratione doloremque quia maiores aut

vero rerum temporibus dolo

ipsa repellendus fugit nis

et doloremque nulla

repellendus sunt dolores architecto voluptatum

ab voluptatum amet voluptas

accusamus eos facilis sint et aut voluptatem

quo laboriosam deleniti aut qu

dolorum est consequatur ea mollitia in culpa

molestiae ipsa aut voluptatibus pariatur dolor nihil

```
FetchExample.jsx:13
```

```
(200) [{"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}, {"..."}]
▶ [0 ... 99]
▶ [100 ... 199]
length: 200
[[Prototype]]: Array(0)
```

fetch()를 사용한 비동기 처리

▪ App.jsx

```
import FetchTodos from './fetch_ex/FetchTodos'
import FetchTodoById from './fetch_ex/FetchTodoById'
import { BrowserRouter, Route, Routes } from 'react-router-dom'

function App() {

  return (
    <>
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<FetchTodos />} />
          <Route path="/:id" element={<FetchTodoById />} />
        </Routes>
      </BrowserRouter>
    </>
  )
}
```

fetch()를 사용한 비동기 처리

▪ App.css

```
.fetch-todos,  
.axios-todos,  
.axios-new-todo {  
  padding: 20px;  
}  
  
.fetch-todo-by-id,  
.axios-todo-by-id {  
  padding: 20px;  
  line-height: 2.5;  
}
```

fetch()를 사용한 비동기 처리

▪ 할 일 목록 – FetchTodos.jsx

```
// JSONPlaceholder에서 사용자 데이터를 가져오는 예시
const FetchTodos = () => {
  const [data, setData] = useState([]);

  // 컴포넌트가 마운트될 때 한 번만 실행
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/todos")
      .then((response) => response.json()) // JSON 응답을 JavaScript 객체로 변환
      .then((result) => {
        setData(result); // 변환된 데이터를 상태에 저장
        console.log(result);
      })
      .catch((error) => console.error(error)); // 오류 처리
  }, []);
}
```

fetch()를 사용한 비동기 처리

▪ 할 일 목록 – FetchTodos.jsx

```
return (  
  <div className="fetch-todos">  
    <h2>할 일 (To-do) 데이터</h2>  
    <ul>  
      {data.map((todo) => (  
        <li key={todo.id}>  
          <Link to={`/${todo.id}`}>{todo.title}</Link>  
        </li>  
      ))}  
    </ul>  
  </div>  
);  
};  
  
export default FetchTodos;
```

fetch()를 사용한 비동기 처리

- 할 일 1건

할 일(To-do) 데이터

제목: delectus aut autem

완료 여부: ☐ 미완료

fetch()를 사용한 비동기 처리

- 할 일 1건 – FetchTodoById.jsx

```
import { useParams } from "react-router-dom";

const FetchTodoById = () => {
  const { id } = useParams(); // URL에서 id 파라미터를 가져옴
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch(`https://jsonplaceholder.typicode.com/todos/${id}`)
      .then((response) => response.json())
      // JSON 응답을 JavaScript 객체로 변환
      .then((result) => {
        setData(result); // 변환된 데이터를 상태에 저장
        console.log(result);
      })
      .catch((error) => console.error(error)); // 오류 처리
  }, [id]); // id가 변경될 때마다 데이터를 다시 가져옴
}
```

fetch()를 사용한 비동기 처리

- 할 일 1건 – FetchTodoById.jsx

```
return (  
  <div className="fetch-todo-by-id">  
    <h2>할 일 (To-do) 데이터</h2>  
    {data && (  
      <div>  
        <p>  
          <strong>제목:</strong> {data.title}  
        </p>  
        <p>  
          <strong>완료 여부:</strong>  
          {data.completed ? "● 완료" : "○ 미완료"}  
        </p>  
      </div>  
    )}  
  </div>  
)  
};
```

axios()를 사용한 비동기 처리

■ axios란?

HTTP 요청을 쉽게 보내기 위한 라이브러리

- 서버(API)와 통신할 때 사용
- GET, POST, PUT, DELETE 요청 처리
- **fetch의 불편한 점** (JSON 변환 직접 해야 함, 에러 처리 불편) **보완**

■ axios 라이브러리 설치

```
npm install axios
```

```
"dependencies": {  
  "axios": "^1.14.0",  
  "react": "^19.2.4",  
  "react-dom": "^19.2.4"  
},
```

axios()를 사용한 비동기 처리

- axios 메서드

메서드	용도	axios 함수
GET	데이터 조회	axios.get()
POST	데이터 생성	axios.post()
PUT	전체 수정	axios.put()
PATCH	일부 수정	axios.patch()
DELETE	데이터 삭제	axios.delete()

axios()를 사용한 비동기 처리

■ 할 일 목록

할 일(To-do) 데이터

delectus aut autem
quis ut nam facilis et officia qui
fugiat veniam minus
et porro tempora
laboriosam mollitia et enim quasi adipisci quia provident illum
qui ullam ratione quibusdam voluptatem quia omnis
illo expedita consequatur quia in
quo adipisci enim quam ut ab
molestiae perspiciatis ipsa
illo est ratione doloremque quia maiores aut
vero rerum temporibus dolor
ipsa repellendus fugit nisi
et doloremque nulla
repellendus sunt dolores architecto voluptatum
ab voluptatum amet voluptas
accusamus eos facilis sint et aut voluptatem
quo laboriosam deleniti aut qui
dolorum est consequatur ea mollitia in culpa
molestiae ipsa aut voluptatibus pariatur dolor nihil
ullam nobis libero sapiente ad optio sint

axios()를 사용한 비동기 처리

▪ App.jsx

```
function App() {  
  return (  
    <>  
      <BrowserRouter>  
        <Routes>  
          { /* Fetch Todo */}  
          <Route path="/" element={<FetchTodos />} />  
          <Route path="/:id" element={<FetchTodoById />} />  
  
          { /* Axios Todo */}  
          <Route path="/axios" element={<AxiosTodos />} />  
          <Route path="/axios/:id" element={<AxiosTodoById />} />  
          <Route path="/axios-post" element={<AxiosPost />} />  
        </Routes>  
      </BrowserRouter>  
    </>  
  )  
}
```

axios()를 사용한 비동기 처리

- 할 일 목록 – AxiosTodos.jsx

```
const AxiosTodos = () => {  
  const [data, setData] = useState([]);  
  
  useEffect(() => {  
    // JSONPlaceholder API에서 할 일 데이터를 가져옵니다.  
    axios.get("https://jsonplaceholder.typicode.com/todos")  
      .then((response) => {  
        setData(response.data); // 가져온 데이터를 상태에 저장  
        console.log(response.data);  
      })  
      .catch((error) => console.error(error)); // 오류 처리  
  }, []);  
}
```

axios()를 사용한 비동기 처리

- 할 일 목록 – AxiosTodos.jsx

```
return (  
  <div>  
    <h2>할 일 (To-do) 데이터</h2>  
    <ul>  
      {data.map((todo) => (  
        <li key={todo.id}>  
          <Link to={`/axios/${todo.id}`}>{todo.title}</Link>  
        </li>  
      ))}  
    </ul>  
  </div>  
);  
}
```

axios()를 사용한 비동기 처리

- 할 일 1건

할 일(To-do) 데이터

제목: delectus aut autem

완료 여부: ☐ 미완료

axios()를 사용한 비동기 처리

- 할 일 1건 – AxiosTodo.jsx

```
const AxiosTodoById = () => {  
  const {id} = useParams(); // URL에서 ID를 가져옵니다.  
  const [data, setData] = useState(null);  
  
  useEffect(() => {  
    // JSONPlaceholder API에서 특정 ID의 할 일 데이터를 가져옵니다.  
    axios.get(`https://jsonplaceholder.typicode.com/todos/${id}`)  
      .then((response) => {  
        setData(response.data); // 가져온 데이터를 상태에 저장  
        console.log(response.data);  
      })  
      .catch((error) => console.error(error));  
  }, [id]); // id가 변경될 때마다 데이터를 다시 가져옴
```

axios()를 사용한 비동기 처리

- 할 일 1건 – AxiosTodo.jsx

```
return (  
  <div className="axios-todo-by-id">  
    <h2>할 일 (To-do) 데이터</h2>  
    {data && (  
      <div>  
        <p>  
          <strong>제목: </strong> {data.title}  
        </p>  
        <p>  
          <strong>완료 여부: </strong>  
          {data.completed ? "● 완료" : "○ 미완료"}  
        </p>  
      </div>  
    )}  
  </div>  
);  
}
```

axios()를 사용한 비동기 처리

- 할 일 추가 – AxiosNewTodo.jsx

할 일(To-do) 추가

생성된 데이터:

```
▼ {title: '따뜻하게 운동하기', completed: false, id: 201} i  
  completed: false  
  id: 201  
  title: "따뜻하게 운동하기"  
  ► [[Prototype]]: Object
```

axios()를 사용한 비동기 처리

▪ 할 일 추가 – AxiosNewTodo.jsx

```
const AxiosNewTodo = () => {  
  // 할 일 제목을 저장하는 상태  
  const [title, setTitle] = useState("");  
  
  // 입력 필드 변경 시 상태 업데이트  
  const handleChange = (e) => {  
    setTitle(e.target.value);  
  }  
  
  // 폼 제출 시 POST 요청을 보내는 함수  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    // JSONPlaceholder API에서 할 일 데이터를 생성합니다.  
    axios.post("https://jsonplaceholder.typicode.com/todos", {  
      title: title,  
      completed: false,  
    })  
      .then((response) => {  
        console.log("생성된 데이터:", response.data);  
        alert("등록 완료!");  
        setTitle(""); // 입력 필드 초기화  
      })  
      .catch((error) => console.error(error));  
  }  
}
```

axios()를 사용한 비동기 처리

- 할 일 추가 – AxiosNewTodo.jsx

```
return (  
  <div className="axios-new-todo">  
    <h2>할 일 (To-do) 추가</h2>  
    <form onSubmit={handleSubmit}>  
      <input  
        type="text"  
        value={title}  
        onChange={handleChange}  
        placeholder="할 일을 입력하세요"  
      />  
      <button type="submit" style={{marginLeft: '6px'}}>등록</button>  
    </form>  
  </div>  
);  
}
```